



数据库系统

胡 敏 教授 博士生导师
合肥工业大学 计算机与信息学院
jsjxhumin@hfut.edu.cn

第8章 数据库编程



厚德 笃学 崇实 尚新

8.1 嵌入式SQL

8.2 过程化SQL

8.3 存储过程和函数

8.4 ODBC编程

***8.5 OLE DB**

***8.6 JDBC编程**

8.7 小结

8.4 ODBC编程

- 8.4.1 数据库互连概述
- 8.4.2 ODBC工作原理概述
- *8.4.3 ODBC API 基础
- 8.4.4 ODBC的工作流程
- 8.4.5 小结

8.4.1 数据库互连概述

- **ODBC: Open DataBase Connection**
- **ODBC**是一种标准---不同语言的**应用程序**与不同**数据库服务器**之间通讯的标准。
- **ODBC产生的原因:**
 - 由于不同的数据库管理系统的存在，在某个RDBMS下编写的应用程序就不能在另一个RDBMS下运行
 - 许多应用程序需要共享多个部门的数据资源，访问不同的RDBMS



8.4.1 数据库互连概述

■ ODBC:

- 是微软公司开放服务体系(Windows Open Services Architecture, WOSA)中有关数据库的一个组成部分
- 提供了一组访问数据库的标准API (应用程序接口), 支持应用程序与数据库服务器的交互

■ 应用程序通过调用ODBC API, 实现

- 与数据服务器的连接
- 向数据库服务器发送SQL命令
- 一条一条的提取数据库检索结果中的元组传递给应用程序的变量

ODBC约束力:

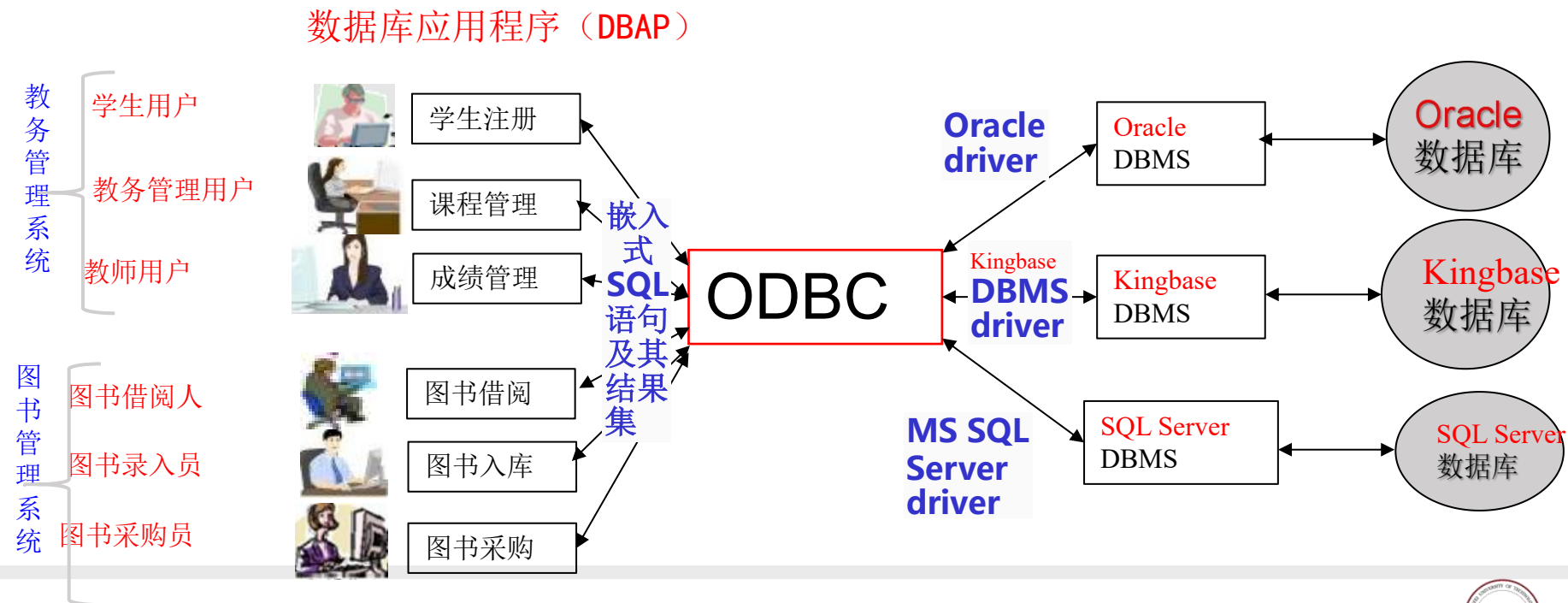
- 规范应用开发
- 规范RDBMS应用接口





数据库互连概述（续）

- 具体的**DBMS**提供一套驱动程序，即**Driver**库函数，供**ODBC**调用，以便实现数据库与应用程序的连接。
- **ODBC**可以配合很多高级语言来使用，如**C, C++, C#, Visual Basic, Power-Builder**等等；



8.4 ODBC编程

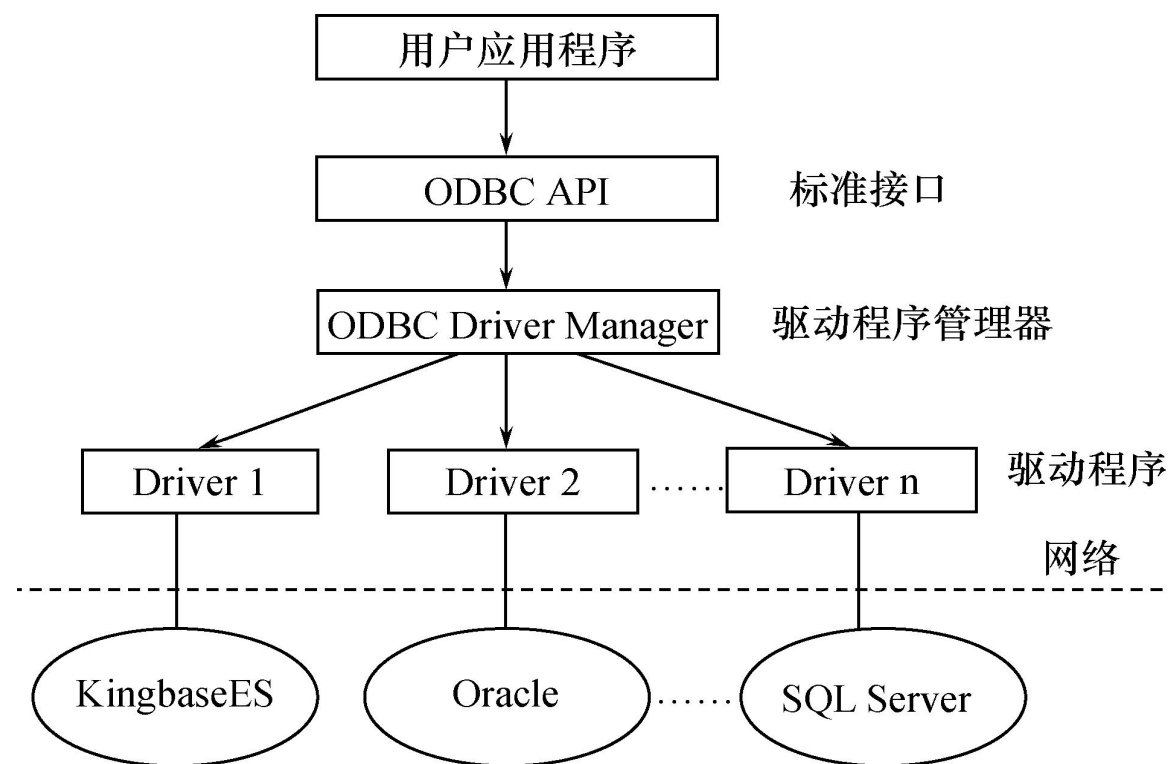
- 8.4.1 数据库互连概述
- 8.4.2 ODBC工作原理概述
- *8.4.3 ODBC API 基础
- 8.4.4 ODBC的工作流程
- 8.4.5 小结



8.4.2 ODBC工作原理概述

- ODBC应用系统的体系结构：

- 一、 用户应用程序
- 二、 驱动程序管理器
- 三、 数据库驱动程序
- 四、 **ODBC**数据源管理





一、应用程序

• ODBC应用程序包括的内容:

- 请求连接数据库;
- 向数据源发送SQL语句;
- 为SQL语句执行结果分配存储空间, 定义所读取的数据格式;
- 获取数据库操作结果, 或处理错误;
- 进行数据处理并向用户提交处理结果;
- 请求事务的提交和回滚操作;
- 断开与数据源的连接。





二、驱动程序管理器

- 驱动程序管理器：用来管理各种驱动程序
 - 包含在ODBC32.DLL中
 - 管理应用程序和驱动程序之间的通信
 - 建立、配置或删除数据源并查看系统当前所安装的数据库ODBC驱动程序
 - 主要功能：
 - 装载ODBC驱动程序
 - 选择和连接正确的驱动程序
 - 管理数据源
 - 检查ODBC调用参数的合法性
 - 记录ODBC函数的调用等





三、数据库驱动程序

- ODBC通过驱动程序来提供应用系统与数据库平台的独立性
- ODBC应用程序不能直接存取数据库
 - 其各种操作请求由驱动程序管理器提交给某个RDBMS的ODBC驱动程序
 - 通过调用驱动程序所支持的函数来存取数据库。
 - 数据库的操作结果也通过驱动程序返回给应用程序。
 - 如果应用程序要操纵不同的数据库，就要动态地链接到不同的驱动程序上。





四、ODBC数据源管理

- 数据源：是最终用户需要访问的数据，包含了数据库位置和数据库类型等信息，是一种数据连接的抽象
- 数据源对最终用户是透明的
 - ODBC给每个被访问的数据源指定唯一的数据源名（Data Source Name，简称DSN），并映射到所有必要的、用来存取数据的低层软件
 - 在连接中，用数据源名来代表用户名、服务器名、所连接的数据库名等
 - 最终用户无需知道DBMS或其他数据管理软件、网络以及有关ODBC驱动程序的细节



例如，假设某个学校在MS SQL Server和KingbaseES上创建了两个数据库：学校人事数据库和教学科研数据库。

- 学校的信息系统要从这两个数据库中存取数据
- 为方便与两个数据库连接，为学校人事数据库创建一个数据源名PERSON，为教学科研数据库创建一个名为EDU的数据源。
- 当要访问每一个数据库时，只要与PERSON和EDU连接即可,不需要记住使用的驱动程序、服务器名称、数据库名

8.4 ODBC编程



- 8.4.1 数据库互连概述
- 8.4.2 ODBC工作原理概述
- ***8.4.3 ODBC API 基础**
- 8.4.4 ODBC的工作流程
- 8.4.5 小结



8.4 ODBC编程



- 8.4.1 数据库互连概述
- 8.4.2 ODBC工作原理概述
- *8.4.3 ODBC API 基础
- 8.4.4 ODBC的工作流程
- 8.4.5 小结



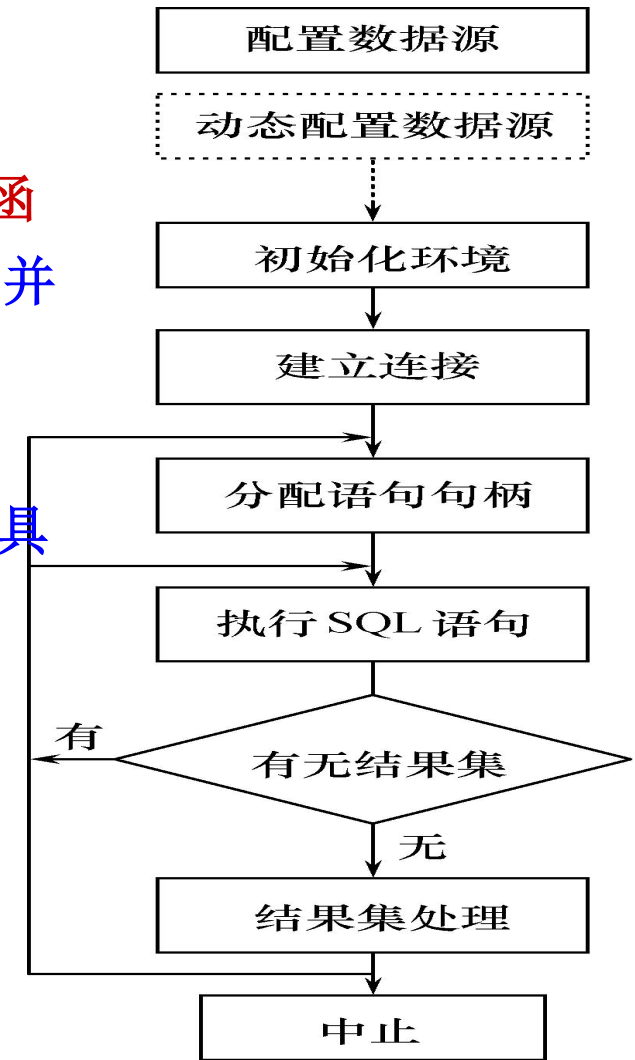


8.4.4 ODBC的工作流程

• ODBC的工作流程:

- **ODBC**应用前, 需要确认具体**DBMS Driver**被安装到**ODBC**环境中
- 当应用程序调用**ODBC API**时, **ODBC API**会调用具体**DBMS Driver**库函数, **DBMS Driver**库函数则与数据库服务器通讯, 执行相应的请求动作并返回检索结果
- **ODBC**应用程序首先要分配一个**SQL**环境, 再产生一个数据库连接句柄
- 应用程序使用**SQLConnect()**, 打开一个数据库连接, **SQLConnect()**的具体参数:

- ✓ **connection handle**, 连接句柄
- ✓ **the server**, 要连接的数据库服务器
- ✓ **the user identifier**, 用户
- ✓ **password**, 密码
- ✓ **SQL_NTS** 类型说明前面的参数是空终止的字符串



[例13] 将KingbaseES数据库中Student表的数据备份到SQL SERVER数据库中。

- 该应用涉及两个不同的RDBMS中的数据源
- 使用ODBC来开发应用程序，只要改变应用程序中连接函数（SQLConnect）的参数，就可以连接不同RDBMS的驱动程序，连接两个数据源

- 在应用程序运行前，已经在KingbaseES和SQL SERVER中分别建立了STUDENT关系表

```
CREATE TABLE Student
```

```
（ Sno CHAR（9） PRIMARY KEY，
```

```
  Sname CHAR（20） UNQUE
```

```
  Ssex CHAR（2），
```

```
  Sage SMALLINT，
```

```
  Sdept CHAR（20）
```

```
）；
```

- 应用程序要执行的操作是：
 - 在KingbaseES上执行SELECT * FROM STUDENT;
 - 把获取的结果集，通过多次执行

```
INSERT INTO STUDENT (Sno, Sname, Ssex, Sage, Sddept) VALUES  
(?, ?, ?, ?, ?);
```

- 插入到SQL SERVER的STUDENT表中

- 操作步骤:

- 一、 配置数据源
- 二、 初始化环境
- 三、 建立连接
- 四、 分配语句句柄
- 五、 执行SQL语句
- 六、 结果集处理
- 七、 中止处理



一、配置数据源

- 配置数据源两种方法：
 - (1)运行数据源管理工具来进行配置；
 - (2)使用Driver Manager 提供的ConfigDsn函数来增加、修改或删除数据源
- 在〔例13〕中，采用了第一种方法创建数据源。因为要同时用到KingbaseES和SQL Server，所以分别建立两个数据源，将其取名为KingbaseES ODBC和SQLServer。





配置数据源（续）

[例13] 创建数据源的详细过程

```
#include <stdlib.h>
#include <stdio.h>
#include <windows.h>
#include <sql.h>
#include <sqlext.h>
#include <Sqltypes.h>
#define SNO_LEN 30
#define NAME_LEN 50
#define DEPART_LEN 100
#define SSEX_LEN 5
```





配置数据源（续）

[例13] 创建数据源---第一步：定义句柄和变量

```
int main()
{
    /* Step 1 定义句柄和变量 */
    //以king开头的表示的是连接KingbaseES的变量
    //以server开头的表示的是连接SQLSERVER的变量
    SQLHENV   kinghenv, serverhenv;           //环境句柄
    SQLHDBC    kinghdbc, serverhdbc;          //连接句柄
    SQLHSTMT   kinghstmt, serverhstmt;       //语句句柄
    SQLRETURN  ret;
    SQLCHAR    sName [NAME_LEN] , sDepart [DEPART_LEN] ,
    sSex [SSEX_LEN] , sSno [SNO_LEN] ;
    SQLINTEGER sAge;
    SQLINTEGER cbAge=0, cbSno=SQL_NTS, cbSex=SQL_NTS,
    cbName=SQL_NTS, cbDepart=SQL_NTS;
```



二、初始化环境

- 没有和具体的驱动程序相关联，由**Driver Manager**来进行控制，并配置环境属性
- 应用程序通过调用连接函数和某个数据源进行连接后，**Driver Manager**才调用所连的驱动程序中的**SQLAllocHandle**，来真正分配环境句柄的数据结构



初始化环境代码

[例13] 创建数据源---第二步：初始化环境

/* Step 2 初始化环境 */

```
ret=SQLAllocHandle(SQL_HANDLE_ENV, SQL_NULL_HANDLE, &kinghenv);  
ret=SQLAllocHandle(SQL_HANDLE_ENV, SQL_NULL_HANDLE, &serverhenv);  
ret=SQLSetEnvAttr(kinghenv,          SQL_ATTR_ODBC_VERSION,  
                  (void*)SQL_OV_ODBC3, 0);  
ret=SQLSetEnvAttr(serverhenv,        SQL_ATTR_ODBC_VERSION,  
                  (void*)SQL_OV_ODBC3, 0);
```





三、 建立连接

- 应用程序调用SQLAllocHandle分配连接句柄，通过SQLConnect、SQLDriverConnect或SQLBrowseConnect与数据源连接
- SQLConnect连接函数，输入参数为：
 - 配置好的数据源名称
 - 用户ID
 - 口令
- [例13]中KingbaseES ODBC为数据源名字，SYSTEM为用户名，MANAGER为用户密码





建立连接代码

[例13] 创建数据源---第三步：建立连接

/* Step 3 :建立连接 */

```
ret=SQLAllocHandle(SQL_HANDLE_DBC, kinghenv, &kinghdbc);
```

```
ret=SQLAllocHandle(SQL_HANDLE_DBC, serverhenv, &serverhdbc);
```

```
ret=SQLConnect(kinghdbc, "KingbaseES ODBC", SQL_NTS, "SYSTEM", SQL_NTS,  
"MANAGER", SQL_NTS);
```

```
if (!SQL_SUCCEEDED(ret))//连接失败时返回错误值
```

```
return-1;
```

```
ret=SQLConnect(serverhdbc, "SQLServer", SQL_NTS, "sa",
```

```
SQL_NTS, "sa", SQL_NTS);
```

```
if (!SQL_SUCCEEDED(ret) ) //连接失败时返回错误值
```

```
return -1;
```





四、分配语句句柄

- 处理任何**SQL**语句之前，应用程序还需要首先分配一个语句句柄
- 语句句柄含有具体的**SQL**语句以及输出的结果集等信息
- [例13]中分配了两个语句句柄：
 - 一个用来从KingbaseES中读取数据产生结果集（kinghstmt）
 - 一个用来向SQLSERVER插入数据（serverhstmt）
- 应用程序还可以通过SQLtStmtAttr来设置语句属性（也可以使用默认值）
- [例13]中结果集绑定的方式为按列绑定





分配语句句柄代码

[例13] 创建数据源---第四步

/* Step 4 :初始化语句句柄 */

```
ret=SQLAllocHandle(SQL_HANDLE_STMT, kinghdbc, &kinghstmt);
```

```
ret=SQLSetStmtAttr(kinghstmt, SQL_ATTR_ROW_BIND_TYPE, (SQLPOINTER)  
    SQL_BIND_BY_COLUMN, SQL_IS_INTEGER );
```

```
ret=SQLAllocHandle(SQL_HANDLE_STMT, serverhdbc, &serverhstmt);
```





五、 执行**SQL**语句

- 应用程序处理SQL语句的两种方式：
 - 预处理（SQLPrepare、SQLExecute适用于语句的多次执行）
 - 直接执行（SQLExecdirect）
- 如果SQL语句含有参数，应用程序为每个参数调用SQLBindParameter，并把它们绑定至应用程序变量
- 应用程序可以直接通过改变应用程序缓冲区的内容从而在程序中动态的改变SQL语句的具体执行



- 应用程序根据语句的类型进行的处理
 - 有结果集的语句（**select**或是编目函数），则进行结果集处理。
 - 没有结果集的函数，可以直接利用本语句句柄继续执行新的语句或是获取行计数（本次执行所影响的行数）之后继续执行。
- 在[例13]中，使用**SQLExecdirect**获取KingbaseES中的结果集，并将结果集根据各列不同的数据类型绑定到用户程序缓冲区
- 在插入数据时，采用了预编译的方式，首先通过**SQLPrepare**来预处理SQL语句，将每一列绑定到用户缓冲区
- 应用程序可以直接修改结果集缓冲区的内容

[例13] 创建数据源---第五步：执行SQL语句

/* Step 5 :两种方式执行语句 */

/* 预编译带有参数的语句 */

```
ret=SQLPrepare(serverhstmt, "INSERT INTO STUDENT(SNO, SNAME, SSEX, SAGE, SDEPT)
VALUES (?, ?, ?, ?, ?)", SQL_NTS);
if (ret==SQL_SUCCESS || ret==SQL_SUCCESS_WITH_INFO)
{
ret=SQLBindParameter(serverhstmt, 1, SQL_PARAM_INPUT, SQL_C_CHAR, SQL_CHAR, SNO_LEN,
0, sSno, 0, &cbSno);
ret=SQLBindParameter(serverhstmt, 2, SQL_PARAM_INPUT, SQL_C_CHAR, SQL_CHAR,
NAME_LEN, 0, sName, 0, &cbName);
ret=SQLBindParameter(serverhstmt, 3, SQL_PARAM_INPUT, SQL_C_CHAR, SQL_CHAR, 2, 0,
sSex, 0, &cbSex);
ret=SQLBindParameter(serverhstmt, 4, SQL_PARAM_INPUT,
SQL_C_LONG, SQL_INTEGER, 0, 0, &sAge, 0, &cbAge);
```

```
ret=SQLBindParameter(serverhstmt, 5, SQL_PARAM_INPUT, SQL_C_CHAR,  
SQL_CHAR, DEPART_LEN, 0, sDepart, 0, &cbDepart);  
}  
/*执行SQL语句*/  
ret=SQLExecDirect(kinghstmt, "SELECT * FROM STUDENT", SQL_NTS);  
if (ret==SQL_SUCCESS || ret==SQL_SUCCESS_WITH_INFO)  
{  
    ret=SQLBindCol(kinghstmt, 1, SQL_C_CHAR, sSno, SNO_LEN, &cbSno);  
    ret=SQLBindCol(kinghstmt, 2, SQL_C_CHAR, sName, NAME_LEN, &cbName);  
    ret=SQLBindCol(kinghstmt, 3, SQL_C_CHAR, sSex, SSEX_LEN, &cbSex);  
    ret=SQLBindCol(kinghstmt, 4, SQL_C_LONG, &sAge, 0, &cbAge);  
    ret=SQLBindCol(kinghstmt, 5, SQL_C_CHAR, sDepart, DEPART_LEN, &cbDepart);  
}
```

六、结果集处理

- 应用程序可以通过SQLNumResultCols来获取结果集中的列数
- 通过SQLDescribeCol或是SQLColAttribute函数来获取结果集每一列的名称、数据类型、精度和范围



结果集处理（续）

- ODBC中使用游标来处理结果集数据
- ODBC中游标类型：
 - forward-only游标，是ODBC的默认游标类型
 - 可滚动（scroll）游标：
 - 静态（static）
 - 动态（dynamic）
 - 码集驱动（keyset-driven）
 - 混合型（mixed）





结果集处理（续）

- 结果集处理步骤：
 - ODBC游标打开方式不同于嵌入式SQL，不是显式声明而是系统自动产生一个游标（Cursor），当结果集刚刚生成时，游标指向第一行数据之前
 - 应用程序通过SQLBindCol，把查询结果绑定到应用程序缓冲区中，通过SQLFetch或是SQLFetchScroll来移动游标获取结果集中的每一行数据
 - 对于如图像这类特别的数据类型当一个缓冲区不足以容纳所有的数据时，可以通过SQLGetData分多次获取
 - 最后通过SQLClosecursor来关闭游标



[例13] 创建数据源---第六步：结果集处理

```
/* Step 6 : 处理结果集并执行预编译后的语句*/
while ( (ret=SQLFetch(kinghstmt) ) !=SQL_NO_DATA_FOUND)
{
    if(ret==SQL_ERROR) printf("Fetch error \n");
    else
        ret=SQLExecute(serverhstmt);
}
```



七、中止处理

- 应用程序中止步骤：
 - 首先释放语句句柄
 - 释放数据库连接
 - 与数据库服务器断开
 - 释放ODBC环境





中止处理代码

[例13] 创建数据源---第七步：中止处理

```
/* Step 7 中止处理*/  
SQLFreeHandle(SQL_HANDLE_STMT, kinghstmt);  
SQLDisconnect(kinghdbc);  
SQLFreeHandle(SQL_HANDLE_DBC, kinghdbc);  
SQLFreeHandle(SQL_HANDLE_ENV, kinghenv);  
SQLFreeHandle(SQL_HANDLE_STMT, serverhstmt);  
SQLDisconnect(serverhdbc);  
SQLFreeHandle(SQL_HANDLE_DBC, serverhdbc);  
SQLFreeHandle(SQL_HANDLE_ENV, serverhenv);  
return 0;  
}
```



8.4 ODBC编程



- 8.4.1 数据库互连概述
- 8.4.2 ODBC工作原理概述
- *8.4.3 ODBC API 基础
- 8.4.4 ODBC的工作流程
- 8.4.5 小结



8.4.5 小结

- ODBC目的：为了提高应用系统与数据库平台的独立性，使得应用系统的移植变得容易
- ODBC优点：
 - 使得应用系统的开发与数据库平台的选择、数据库设计等工作并行进行
 - 方便移植
 - 大大缩短整个系统的开发时间

8.1 嵌入式SQL

8.2 过程化SQL

8.3 存储过程和函数

8.4 ODBC编程

***8.5 OLE DB**

8.6 JDBC编程

8.7 小结

8.6 JDBC编程

- 8.6.1 JDBC简介
- 8.6.2 JDBC功能
- 8.6.3 JDBC的编程实施过程
- 8.4.5 小结



8.6.1 JDBC简介

- **JDBC: Java DataBase Connection**

➤ JDBC是一组Java版的应用程序接口API，提供了Java应用程序与数据库服务器的连接和通讯能力。

- **JDBC API**

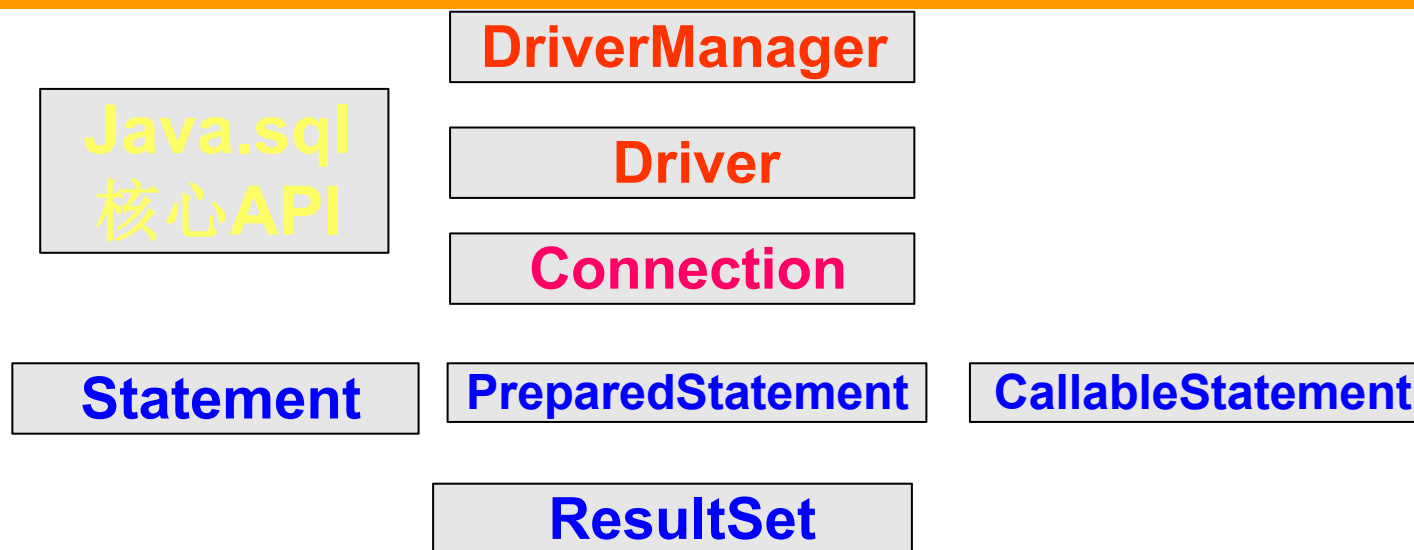
➤ JDBC API 分成两个程序包：

- ✓ **Java.sql 核心API** --J2SE(Java2标准版)的一部分。使用**java.sql.DriverManager**类、**java.sql.Driver**和**java.sql.Connection**接口连接到数据库
- ✓ **Javax.sql 可选扩展API**--J2EE(Java2企业版)的一部分。包含了基于JNDI(Java Naming and Directory Interface, Java命名和目录接口)的资源，以及管理**连接池**、**分布式事务**等，使用**DataSource**接口连接到数据库。





8.6.2 JDBC功能



- **java.sql.DriverManager**——处理驱动的调入并且对产生新数据库连接提供支持
- **Java.sql.Driver**——通过驱动进行数据库访问，连接到数据库的应用程序必须具备 该数据库的特定驱动。
- **java.sql.Connection**——代表对特定数据库的连接。
- **Try {...} Catch {...}** ——异常捕获及其处理
- **java.sql.Statement**——对特定的数据库执行**SQL**语句
- **java.sql.PreparedStatement** —— 用于执行预编译的**SQL**语句
- **java.sql.CallableStatement** ——用于执行对数据库内嵌过程的调用。
- **java.sql.ResultSet**——从当前执行的**SQL**语句中返回结果数据。





8.6.3 JDBC的编程实施过程

- 概念性的基本过程

打开一个连接；创建“Statement”对象，并设置查询语句；使用Statement对象执行 查询，发送查询给数据库服务器和返回结果给应用程序；处理错误的例外机制

- 具体实施过程

- 传递一个Driver给DriverManager，加载数据库驱动。

- ✓ `Class.forName()`

- 通过URL得到一个Connection对象，建立数据库连接

- ✓ `DriverManager.getConnection(sDBUrl)`

- ✓ `•DriverManager.getConnection(sDBUrl,sDBUserID,sDBPassword)`

- 接着创建一个Statement对象(PreparedStatement或CallableStatement)，用来 查询或者修改数据库。

- ✓ `Statement stmt=con.createStatement()`

- 查询返回一个ResultSet。

- ✓ `ResultSet rs=stmt.executeQuery(sSQL)`





8.6.3 JDBC的编程实施过程

示例

```
public static void JDBCexample(String dbid, String userid, String passwd)
```

```
{ try                //错误捕获
{
    Class.forName ("oracle.jdbc.driver.OracleDriver"); Connection conn =
    DriverManager.getConnection(
        "jdbc:oracle:thin:@db.yale.edu:1521:univdb", userid, passwd);
    //加载数据库驱动，建立数据库连接
    Statement stmt = conn.createStatement();
    //创建一个语句对象
    ... Do Actual Work ....
    //进行SQL语句的执行与处理工作
    stmt.close();
    conn.close();
    //关闭语句对象，关闭连接
} catch (SQLException sqle)
{ System.out.println("SQLException : " + sqle);}
}
```





8.6.3 JDBC的编程实施过程

Update to database

```
try { stmt.executeUpdate( "insert into instructor values  
                        ('77987', 'Kim', 'Physics',98000)");  
  
    //插入一条记录  
  
} catch (SQLException sqle) {  
    System.out.println("Could not insert tuple. " + sqle); }
```

Execute query and fetch and print results

```
ResultSet rset = stmt.executeQuery( "select dept_name, avg(salary)" +  
    " from instructor group by dept_name" );  
  
while ( rset.next() ) {  
    System.out.println(rset.getString("dept_name") + "  
    "  
    + rset.getFloat(2)); }
```

//执行一条SQL语句。获取下一条记录。提取“dept_name”属性值，提取第2列即“平均工资”列的值





8.6.3 JDBC的编程实施过程

完整的示例程序

```
public static void JDBCexample(String dbid, String userid, String passwd)
{
    try {
        Class.forName ("oracle.jdbc.driver.OracleDriver");
        Connection conn = DriverManager.getConnection( "jdbc:oracle:thin:
            @db.yale.edu:1521:univdb", userid, passwd); Statement stmt =
        conn.createStatement();
        try {
            stmt.executeUpdate( "insert into instructor values ('77987', 'Kim',
                'Physics',98000)");
        } catch (SQLException sqle) { System.out.println("Could not insert tuple. " + sqle);
        }

        ResultSet rset = stmt.executeQuery( "select dept_name, avg(salary)" + " from instructor group by
            dept_name");
        while ( rset.next() ) {
            System.out.println(rset.getString("dept_name") + " " + rset.getFloat(2)); } stmt.close();
        conn.close();
    } catch (SQLException sqle) { System.out.println("SQLException : " + sqle);
    }
}
```



作业



计算机与信息学院
SCHOOL OF COMPUTER SCIENCE AND INFORMATION ENGINEERING
人工智能学院
SCHOOL OF ARTIFICIAL INTELLIGENCE

- P270: 2



To be continued
Practice makes perfect!



厚德

笃学

崇实

尚新